Reflection and Traceability Report on SFWRENG 4G06 - Capstone Design Project

Team #7, Wardens of the Wild
Felix Hurst
Marcos Hernandez-Rivero
BoWen Liu
Andy Liang

April 6, 2026

[Reflection is an important component of getting the full benefits from a learning experience. Besides the intrinsic benefits of reflection, this document will be used to help the TAs grade how well your team responded to feedback. Therefore, traceability between Revision 0 and Revision 1 is and important part of the reflection exercise. In addition, several CEAB (Canadian Engineering Accreditation Board) Learning Outcomes (LOs) will be assessed based on your reflections. —TPLT]

1 Changes in Response to Feedback

[Summarize the changes made over the course of the project in response to feedback from TAs, the instructor, teammates, other teams, the project supervisor (if present), and from user testers. —TPLT]

[For those teams with an external supervisor, please highlight how the feedback from the supervisor shaped your project. In particular, you should highlight the supervisor's response to your Rev 0 demonstration to them. —TPLT]

[Version control can make the summary relatively easy, if you used issues and meaningful commits. If your feedback is in an issue, and you responded in the issue tracker, you can point to the issue as part of explaining your changes. If addressing the issue required changes to code or documentation, you can point to the specific commit that made the changes. Although the links are helpful for the details, you should include a label for each item of feedback so that the reader has an idea of what each item is about without the need to click on everything to find out. —TPLT]

[If you were not organized with your commits, traceability between feedback and commits will not be feasible to capture after the fact. You will instead need to spend time writing down a summary of the changes made in response to each item of feedback. —TPLT]

[You should address EVERY item of feedback. A table or itemized list is recommended. You should record every item of feedback, along with the source of that feedback and the change you made in response to that feedback. The response can be a change to your documentation, code, or development process. The response can also be the reason why no changes were made in response to the feedback. To make this information manageable, you will record the feedback and response separately for each deliverable in the sections that follow. —TPLT]

[If the feedback is general or incomplete, the TA (or instructor) will not be able to grade your response to feedback. In that case your grade on this document, and likely the Revision 1 versions of the other documents will be low. —TPLT]

1.1 SRS and Hazard Analysis

1.2 Design and Design Documentation

1.3 VnV Plan and Report

2 Extras

In this section, we will summarize this project's two extra deliverables.

2.1 Usability Report

The Usability Report summarized feedback from beta testers of our product. We distributed beta builds of our project to people interested, gave them basic instructions on how to launch the game, asked them to play through the game to the end, and then to submit a feedback survey. The questions on the feedback survey are listed in the VnV Plan document. Two rounds of usability testing were conducted: One in early March, and one in early April. These two rounds allowed us to assess major points of success and failure of our project, collect suggestions on potential improvements, and then measure whether we succeeded in implementing those improvements.

Due to TA feedback, improvements were made to add more detail to the content of the report itself, including the exact number of participants of each survey, more of the suggestions given to our team by survey participants, and further explanations of why our builds lacked certain features.

2.2 Technical Report on the Slime Mold Algorithm

TODO by Andy

3 Design Iteration (LO11 (PrototypeIterate))

Our design began with the idea of the entire world possibly being destructible through the usage of tools like a shovel and rifle. Destroying terrain would turn it into very small particle-like chunks of debris. In addition, we had concepts for a plant representing the slime mold, such as a vine. The player could interact with this vine primarily through destroying terrain in such a way that moved existing sources of water. The vine would then be attracted to the water, growing, building a bridge that crosses a gap, and allowing the player to proceed further through the level. The thought behind this was that a vine would be more visually appealing compared to slime, and a vine's tendril growth could still somewhat accurately represent the growth of a slime mold.

In the early phases of design, we suffered from scope creep. Due to the open-ended nature of our project, there are an infinite number of wildly different ways we could have implemented a solution for our stakeholders. The only thing that truly stayed the same from start to finish, was the notion of this project being a 2D pixel art puzzle platformer showcasing the slime mold algorithm in some way.

However, we spent too much time generating ideas for game mechanics, such as the player's abilities, the types of obstacles in the way, and the method of representing the slime mold. We did not spend enough time settling on what to actually move forward with in these early stages.

Eventually, through discussions within our team as well as with our supervisors, we were able to settle on a core set of features. These core features included:

- The player would have three tools: A water shooter, rifle (impact rounds), and wind fan. The player no longer has a shovel tool, as that tool's functionality became obsolete.

  The water shooter helps the player attract the slime mold.

  The impact rounds can break brittle objects.

  The wind fan can blow slime mold spores.

- The slime mold is now represented accurately in terms of what they look like and do in real life, except in a pixel art style. They are no longer modelled by a plant.

- The world is no longer entirely destructible. Instead, only certain objects can be damaged and destroyed, some only by the player, and some only by the slime mold.

These changes were done primarily in accordance with wishes from our supervisors.

First, we realized it would be best to represent the slime mold closely to what they look like and do in real life, as it is important to our project to showcase slime mold and why it is so special. It would not do to have a plant model

the slime mold instead, even if a plant is more visually appealing in a general sense, compared to slime. Instead of building bridges, the slime became capable of decomposing wood to clear it out of the player's path, and to coat walls to allow the player to climb up.

Second, we determined it would not be feasible for the entire world to be destructible, as it would defeat the purpose of requiring the player to interact with the slime mold to overcome obstacles. If players could just break everything in their way, they might ignore the slime mold completely. Therefore, we made most environmental terrain indestructible, and only certain objects can be destroyed directly by the player. Additionally, wood obstacles can only be destroyed by the slime mold, so this serves to force players to interact with the slime mold.

Third, our supervisors helped us come up with ideas for game mechanics, such as the concept of blowing slime mold spores from one side of an area to another, to help them grow into new slime molds, demonstrating the organism's full life cycle. We also opted to give the player a way to spawn water instead of relying on water from the environment, through the water shooter tool. This gives the player a bit more control over what they can do to help the slime mold, though they still do not have full control over the slime mold. This lack of full control over the slime mold is also important, as it teaches the player an important lesson about consent.

From this, we were able to refine the details of the interactions between the player, the obstacles, the slime mold, and the user interface. We developed a story with clear objectives for the player to follow, so they would understand their place in this fictional world.

In addition to discussion with our supervisors, we also conducted usability testing. This allowed us to get a better picture of what our end users felt about our product. For more details on the Usability Report, see Section 2.1. In summary, survey participants were impressed by the game's aesthetic and visuals, but frustrated by its lack of clarity about game mechanics, what the player could do, and what they were supposed to do to overcome each puzzle. We took this feedback to heart, and immediately began implementing much more UI elements to explain player state, teach them the basics, and give them hints on how to proceed. One major feature involved in this is the journal logs. These interactable objects provide additional context and tips to the player, while still feeling like they are part of the story and experience.

4 Design Decisions (LO12)

In this section, we will reflect on and justify our design decisions, considering the influences of limitations, assumptions, and constraints.

4.1 Limitations

Our primary limitations came from our choice of game engine, Unity. For instance, one of our early design ideas from our supervisors was the concept of having additional input devices. Not just a keyboard and mouse, but also a microphone, and sensors that could detect atmospheric conditions such as temperature and humidity. Unfortunately, only the microphone concept was able to be implemented in the final product. Other kinds of sensors do not have an API that Unity can easily interface with. We scrapped the idea of having atmospheric sensors, instead opting for a product that anyone could play at home with common peripherals.

4.2 Assumptions

We had to make the assumption that most of our audience and potential end users would have a device of sufficient power to run Unity-made software, and that it would be running one of our three supported platforms: Windows, Mac, or Linux. We also assumed that our end users would likely have a keyboard and mouse with which to play, but not likely a game controller. For these reasons, we set up a control scheme for the game that is catered to PC keyboard and mouse setups. We included keyboard and mouse inputs, but not game controller inputs, nor any kind of touchscreen input.

Additionally, our assumptions on end user device performance influenced how much effort we put into optimizing performance on our project's end. Our team owns laptops designed primarily for office work rather than playing games, which we used to test the performance of our product. We aimed for a stable 30 frames per second on these laptops, and assumed that the performance would be similar for most end users. However, ultimately, it will still depend on each person's device, and we cannot guarantee a smooth experience for people with older devices.

4.3 Constraints

Our primary constraint was time. Much of our project's planned content had to be trimmed, condensed, or scrapped, in order to keep up with the deadlines of the Capstone course. This included:

- Additional levels with new puzzles to overcome and more depth to the story
- Additional behavioural traits of the slime mold
- Background music for every level with a shifting emotional tone over time
- A way to save and load progress

In the end, we were happy with the levels we created, the story we told, the slime mold features we presented, and the two songs that play at the beginning

and end of the game. The game ended up being short enough that it is not a huge issue that there is no saving system, either.

5 Economic Considerations (LO23)

There is currently a booming market for AA and indie games developed by limited team sizes. As legacy studios grow in size and bureaucracy and shareholder interests prevail, there is a growing demand by consumers for games which are unique in not only game direction, but as well as artstyle and concept, which is why we believe our game is perfectly positioned in this industry. Due to our team's small size and non-traditional working environment, we estimate the cost of our game to be under $20,000 in terms of possible third party assets, although the diverse skill sets of our members (scripting, art, sound, music, etc.) mitigates the need to outsource aspects of our game. If our team remains a volunteer development team, the staffing cost could be proportional to revenue gained. Marketing cost can be lowered to a minimum by grassroots promotions, such as major content creators reviewing the product for free in exchange for engaging content. Marketing could also simultaneously be done via short form content and developer logs, which are free to make and publish on social media sites. We are anticipated to price our game at around $7.99, which is the current average price for games within average market price of our scope and genre (indie pixel art platformer tags) [1]. If simply offsetting our game development cost, as well as accounting for each member's salary as compared with Talent.com averages ($117,000 in Canada) [2], our game needs to sell around (($20,000 + ($117,000*4))/$7.99) = $61,076 copies. According to Valve, in 2025, around 5863 games earned over $100,000 out of around 40,000 games (including AI-made and low effort games) [3]. This puts our goals at a difficult to attain threshold, however, if the overall gameplay loop is engaging, and time and effort is put into polish and game direction, even one-man teams are able to surpass $100,000 (Stardew Valley, Hollow Knight, etc.) [1]. As mentioned above, there is already a considerable market for games in our genre and scope, and by leveraging social media and grassroots marketing, we believe there will be a healthy audience base for our upcoming game.

6 Reflection on Project Management (LO24)

This section will reflect on processes and tools used for project management.

6.1 How Does Your Project Management Compare to Your Development Plan

We followed some aspects of our development plan, but other aspects were ignored and forgotten over time. Most team members showed up to the majority of our meetings, whether it was a team meeting, TA meeting, or supervisor

meeting. Team members were generally communicative and responsive in our Discord server. We effectively used polls to vote on matters we disagreed on, breaking ties randomly when needed. For most of the project, we remained respectful and professional with one another, except for a few incidents, which we resolved through conflict resolution with the instructor and TA's assistance. Team members had some roles defined early on in the project, however, these were not closely adhered to. The roles ended up shifting over time. In terms of our workflow plan, we only began developing with a steady pace, as a full team, beginning in February 2026, when we introduced weekly sprints and began tracking each team member's tasks more clearly and closely. We tried using a Kanban board on our GitHub repository for this at first, but stopped using it after a while, as it was much easier to track things solely in our Discord server.

We did use the Unity game engine throughout the whole project as we anticipated. However, we struggled to get unit testing to work with GitHub Actions, so we had to pivot to implementing code style formatting for our CI/CD element.

6.2 What Went Well?

The team leader, Felix Hurst, prepared agendas for most meetings, and kept detailed notes of what was shared, discussed and agreed on during meetings.

We had a good way of keeping track of who was contributing to the project and who was not, recording meeting absences and other incidents for the Team Productivity Report. We contacted the instructor when appropriate to help resolve major incidents.

Our sprint system from February onward helped us keep a relatively steady pace of developments in the final stretch of the Capstone course.

We have also kept up steady and healthy communications with our supervisors throughout the course to keep them updated on our progress and get their help with making decisions or generating new ideas.

6.3 What Went Wrong?

One of the biggest issues we had in the first half of this course was that we lacked a rule or agreement to frequently share progress in the form of actual commits to the source code in the repository. This led to a lot of trust being placed on certain team members to complete a task on time, with other teammates being unable to assist as they did not have even a work-in-progress version of the code to look at. It also meant that, after the code was finally pushed to the repository, it would take extra time for the rest of the team to read through it and understand how it worked and how it could be modified, improved and interfaced with. This resulted in significant setbacks.

Additionally, we initially relied on Google Docs to prepare our documentation, when we should have been using Latex from the start. This would have prevented us from having formatting issues with our documents caused by lack of experience with the typesetting system.

6.4 What Would you Do Differently Next Time?

Next time, we would need to set clearer expectations for a steady and consistent pace of development work, and stricter punishments for failing to meet these expectations. It would not be extremely strict, for instance, we would still have allowable excuses for missed deadlines, but those must be communicated ahead of time if possible, with alternative arrangements in place for other teammates to pick up on the unfinished work if necessary, or a promise to catch up after the fact. These rules should be enforced throughout the whole term, not having us slowly let more and more violations slide over time.

Additionally, it likely would have been effective to employ some team building exercises early on in the project's life span, such as some hangout activities in person. Our team did not end up hanging out in person, as half of our team lived far from campus and found it cumbersome to travel such a long distance unless absolutely necessary. However, these kinds of activities could have sparked a friendship among our team members and served to increase productivity as a result, as we would not want to let each other down.

7 Ethics and Equity (LO18)

Our game design is rooted in equity for all prospective players, as well as the game development community as a whole. Our team worked hard to ensure playability for players with baseline skill in games of our similar genre, as well as made sure control schemes are intuitive for our levels and players' expectations. Additionally, we are resolving to implement graphics settings to ensure more players of different hardware processing power are able to enjoy the game as well. On the game development community side, we are ensuring no assets in the final game are the product of generative AI, in order to ensure copyright integrity in the industry.

8 Reflection on Capstone

Our team learned a considerable amount throughout the development of our project. For many of us, it was the first time developing a game within a game engine editor, and as such, there were many things to learn and improve on during the development process. Our team gained experience in technical areas such as compute shaders, high to low level instruction compiling for optimization, runtime object geometry modification, data caching, and much more. Setting up animations and building levels with C# script modules were also a major learning area for our team as well. On the non-technical side, whilst our team had members who were experienced in visual and audio design, integration with our game levels and module mechanics provided us with new learning challenges as we navigated both the technical and non-technical side.

8.1 Which Courses Were Relevant

Courses that were relevant in this course included 2AA4 as well as 4HC3. These courses built and improved our planning and problem-solving skills, especially with the technically demanding modules in our game, encouraging us to tackle a difficult problem iteratively, step by step. Furthermore, 4HC3 taught us the fundamentals of how users interact with software, and thus, how we can improve our UI/UX designs to be more intuitive and enjoyable for the user. Lastly, as these two courses involved group work tasks, it built our teamwork skills in how to delegate and manage progress of deliverables, including sprint planning and task delegation.

8.2 Knowledge/Skills Outside of Courses

Our team utilized our team members' skills outside of the software development courses, such as visual and sound asset creation, as well as Unity game engine knowledge. Visual and sound assets needed to align with our C# script logic and vice versa. Our Unity game also required hooking up scripts and setting up animations and scenes, which is done in the Unity game engine editor.

References

[1] SteamDB, "Steam database." [Online]. Available: https://steamdb.info/

[2] Talent.com, "Game developer salary in canada - average salary." [Online]. Available: https://ca.talent.com/salary?job=game+developer

[3] R. Stanton, "Valve says almost 6,000 games made over $100k on steam last year," *PC Gamer*, 2026. [Online]. Available: https://www.pcgamer.com/gaming-industry/valve-says-over-5000-games-made-over-usd100k-on-steam-last-year/